

软件安全第八次实验报告

姓名：林晖鹏 学号：2312966

一、实验名称：

程序插桩以及HOOK实验

二、实验要求：

复现实验一，基于 Windows MyPinTool 或者 Kali 中复现 malloctrace 这个 PinTool，理解 Pin 插桩工具的核心步骤和相关 API，关注 malloc 和 free 函数的输入输出信息。

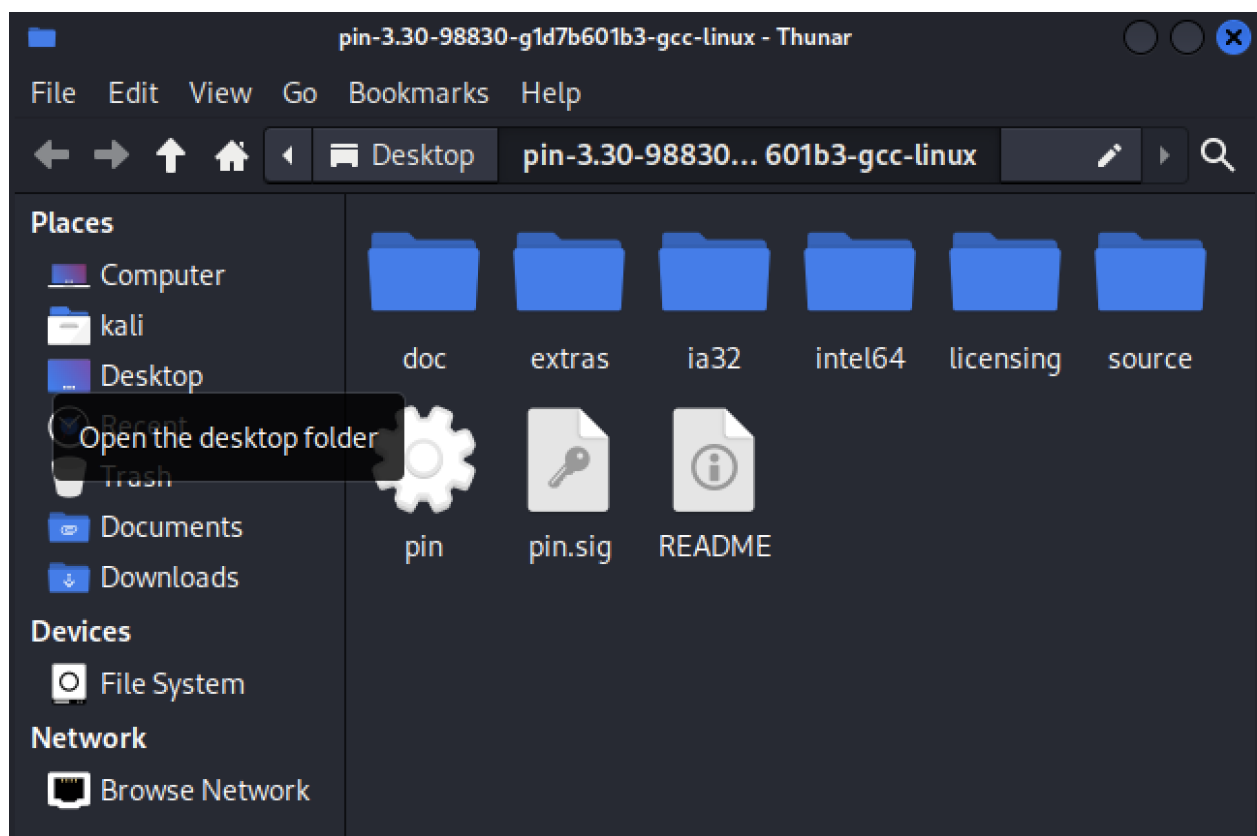
三、实验过程：

1. 在Kali上安装Pintool

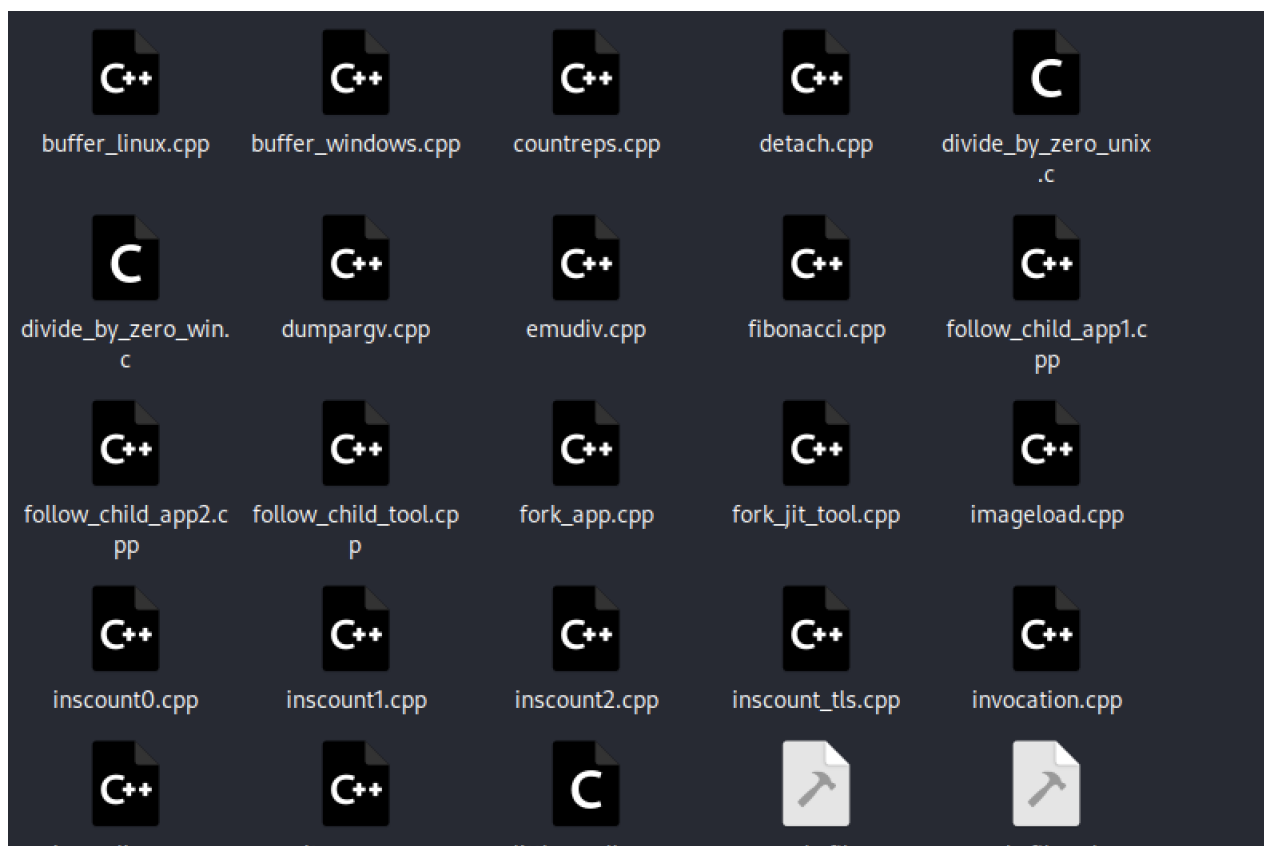
安装网址为 <https://www.intel.com/content/www/us/en/developer/articles/tool/pin-a-binary-instrumentation-tool-downloads.html>。由于虚拟机里面没有装梯子，我们在外面下载后传进去。

这里我们下载的是3.30版本的pintool。

下载传入kali虚拟机解压，我们得到这样的文件夹。



可以看到source/tools/ManualExamples中包含了很多pintool。



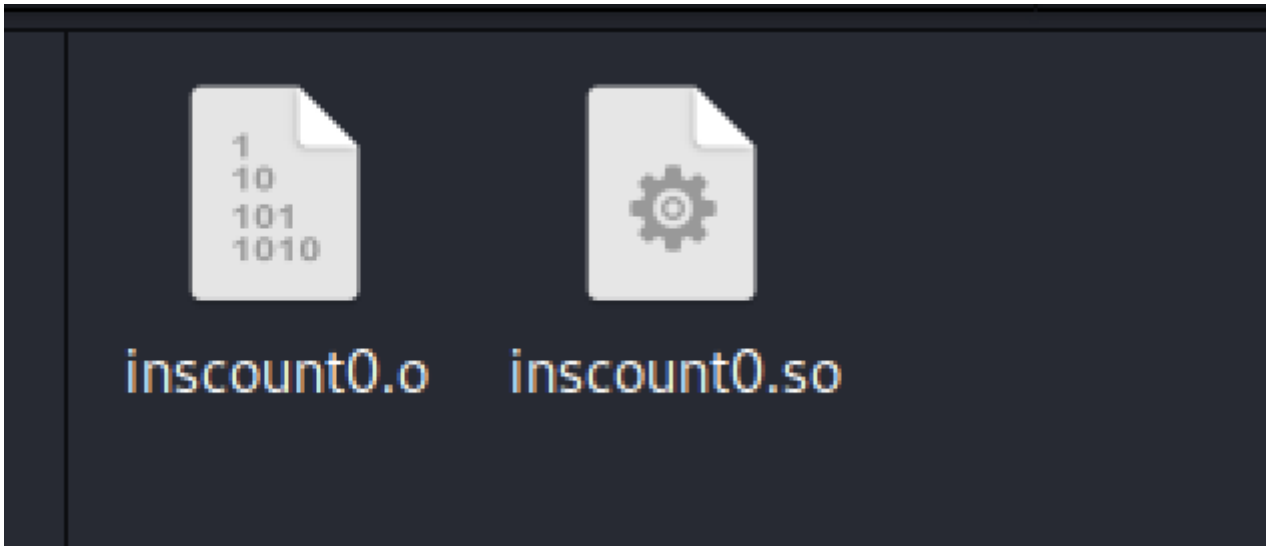
2. Pintool: malloctrace的使用

2.1 编译inscount0.cpp

首先,进入 `source/tools/ManualExamples` ,对 `inscount0.cpp` 进行编译产生其对应的动态链接库。进入该目录后,执行以下指令:

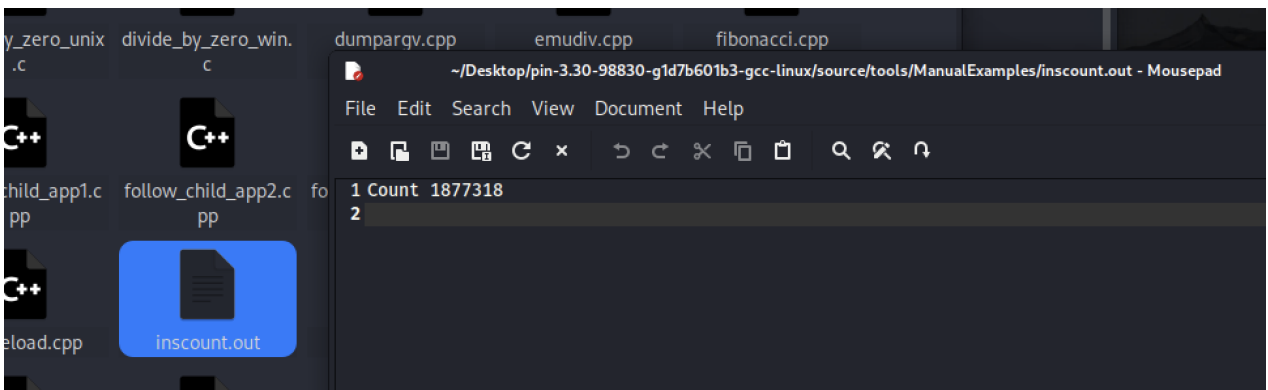
代码块

```
1 make inscount0.test TARGET=intel64
```



编译完成后,会多出一个文件夹,里面有编译好的文件。

外边还有一个文件可以查看编译所运行的指令数:



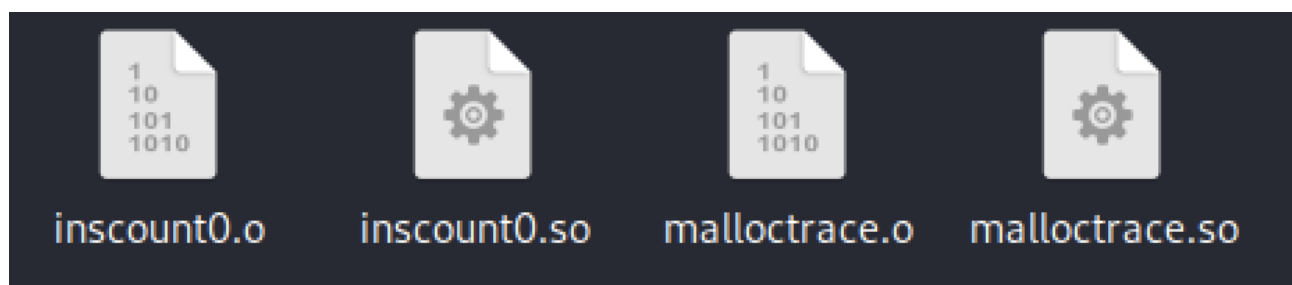
2.2 编译malloctrace.cpp

然后我们继续在这个目录下编译 `malloctrace.cpp` ,获取对应的动态链接库,执行以下指令:

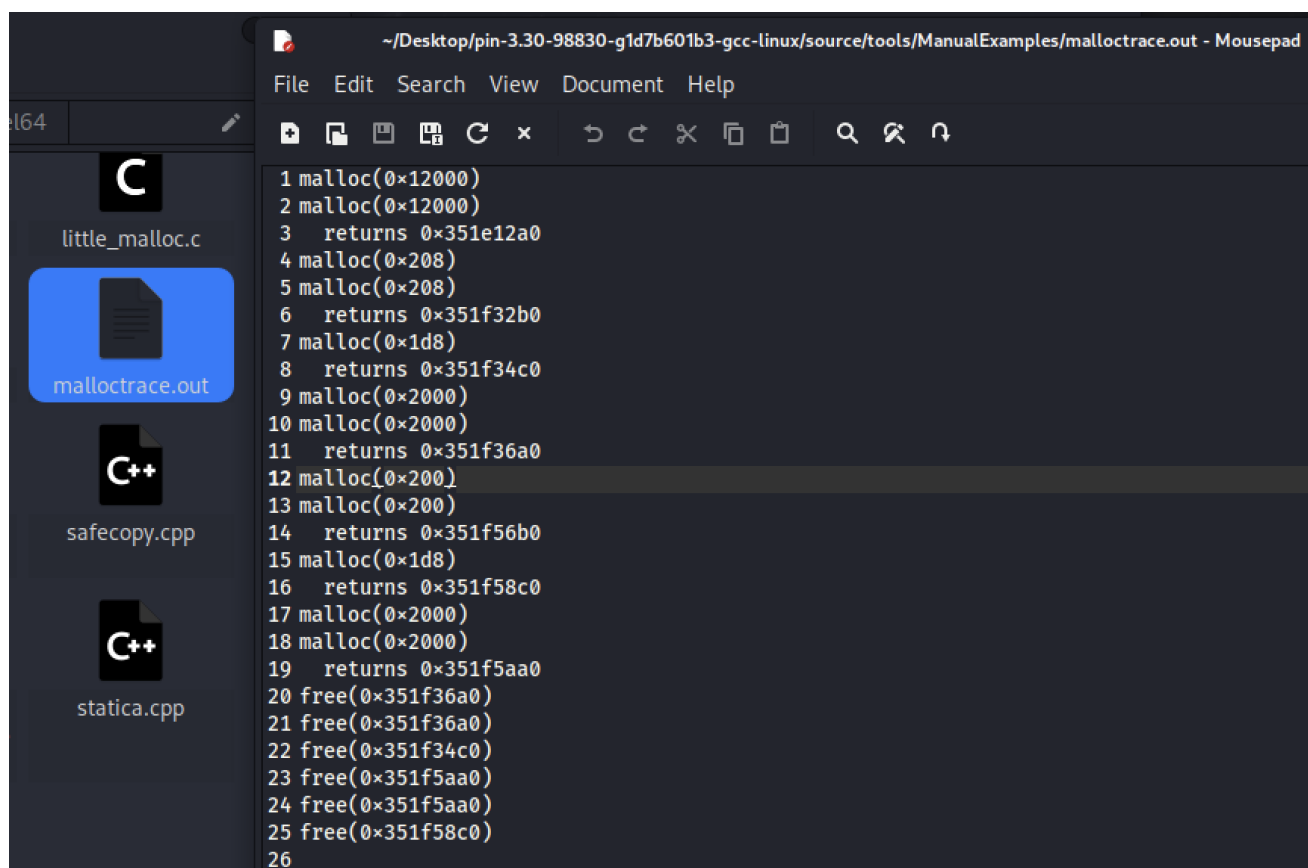
代码块

```
1 make malloctrace.test TARGET=intel64
```

```
(kali@kali)-[~/pin-3.30-98830-g1d7b601b3-gcc-linux/source/tools/ManualExamples]
$ make malloctrace.test TARGET=intel64
g++ -Wall -Werror -Wno-unknown-pragmas -DPIN_CRT=1 -fno-stack-protector -fno-exceptions -funwind-tables -fasynchronous-unwind-tables -fno-rtti -DTARGET_IA32E -DHOST_IA32E -fPIC -DTARGET_LINUX -fabi-version=2 -faligned-new -I../source/include/pin -I../source/include/pin/gen -isystem /home/kali/Desktop/pin-3.30-98830-g1d7b601b3-gcc-linux/extras/cxx/include -isystem /home/kali/Desktop/pin-3.30-98830-g1d7b601b3-gcc-linux/extras/crt/include -isystem /home/kali/Desktop/pin-3.30-98830-g1d7b601b3-gcc-linux/extras/crt/include/kernel/uapi -isystem /home/kali/Desktop/pin-3.30-98830-g1d7b601b3-gcc-linux/extras/crt/include/kernel/uapi/asm-x86 -I../extras/components/include -I../extras/xed-intel64/include/xed -I../source/tools/Utils -I../source/tools/InstLib -O3 -fomit-frame-pointer -fno-strict-aliasing -Wno-dangling-pointer -c -o obj-intel64/malloctrace.o malloctrace.cpp
g++ -shared -Wl,--hash-style=sysv ../runtime/pincrt/crtbeginS.o -Wl,-Bsymbolic -Wl,--version-script=../source/include/pin/pintool.ver -fabi-version=2 -o obj-intel64/malloctrace.so obj-intel64/malloctrace.o -L../runtime/intel64/runtime/pincrt -L../intel64/lib -L../intel64/lib-ext -L../extras/xed-intel64/lib -lpin -lxd ../intel64/runtime/pincrt/crtendS.o -lpindwarf -ldwarf -ldl -dynamic -nostdlib -lc++ -lc++abi -lm -dynamic -luwind-dynamic
/usr/bin/ld: warning: util_host_ia32e.spp.o: missing .note.GNU-stack section implies executable stack
/usr/bin/ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
../pin -t obj-intel64/malloctrace.so -- ../source/tools/Utils/obj-intel64/cp-pin.exe makefile obj-intel64/malloctrace.makefile.copy \
> obj-intel64/malloctrace.out 2>&1
cmp makefile obj-intel64/malloctrace.makefile.copy
rm obj-intel64/malloctrace.makefile.copy
rm obj-intel64/malloctrace.out
```



同时生成了out文件：



2.3 malloc 和 free 函数

我们来关注这两个函数

- malloc:

这个函数是用来申请内存的，括号里面的是申请内存的大小，而 **returns** 后面跟的是这片申请的内存的地址。

- **free:**

这个函数用来释放之前申请的内存，括号里的是内存的地址。

代码块

```
1  malloc(0x12000)
2  malloc(0x12000)
3      returns 0x351e12a0
4  malloc(0x208)
5  malloc(0x208)
6      returns 0x351f32b0
7  malloc(0x1d8)
8      returns 0x351f34c0
9  malloc(0x2000)
10 malloc(0x2000)
11      returns 0x351f36a0
12 malloc(0x200)
13 malloc(0x200)
14      returns 0x351f56b0
15 malloc(0x1d8)
16      returns 0x351f58c0
17 malloc(0x2000)
18 malloc(0x2000)
19      returns 0x351f5aa0
20 free(0x351f36a0)
21 free(0x351f36a0)
22 free(0x351f34c0)
23 free(0x351f5aa0)
24 free(0x351f5aa0)
25 free(0x351f58c0)
```

2.4 插桩实验

完成前面的基础工作，我们开始进行插桩实验。

我们编写一个简单的控制台命令程序 **FirsiC.c**，程序代码如下：

代码块

```
1  #include<stdio.h>
2  void main(){
3      printf("hello world!");
4  }
```

然后编译这个c文件，指令为： `gcc -o First FirstC.c`

编译完成之后，我们执行下面指令用 `inscount0.so` 和 `malloctrace.so` 来对这个执行文件插桩。

代码块

```
1 ./pin -t ./source/tools/ManualExamples/obj-intel64/inscount0.so --  
  '/home/kali/Desktop/First'  
2 ./pin -t ./source/tools/ManualExamples/obj-intel64/malloctrace.so --  
  '/home/kali/Desktop/First'
```

```
(kali㉿kali)-[~/Desktop/pin-3.30-98830-g1d7b601b3-gcc-linux]  
$ ./pin -t ./source/tools/ManualExamples/obj-intel64/malloctrace.so -- '/home/kali/Desktop/First'  
hello world!
```

这里成功打印了 `hello world!`，说明插桩成功。

可以通过out文件查看内存使用以及运行指令数。

```
1 malloc(0x400)  
2 returns 0x55f42aeab2a0  
3 |
```

```
1 Count 124942  
2
```

3. Pintool的基本框架分析

接下来我们分析一下 `malloctrace.cpp` 来分析Pintool的基本框架

- `Arg1Before`：在调用malloc或free之前执行，用于记录函数名称和参数。
- `MallocAfter`：在调用malloc之后执行，记录返回地址。
- `Image`：是用于在程序映像中找到并插桩malloc和free函数的代码。
- `Fini`：在Pin工具完成后关闭输出文件。

代码块

```
1 /*  
2  * Copyright (C) 2004-2021 Intel Corporation.  
3  * SPDX-License-Identifier: MIT  
4  */  
5  
6 #include "pin.H"  
7 #include <iostream>  
8 #include <fstream>  
9 using std::cerr;  
10 using std::endl;  
11 using std::hex;  
12 using std::ios;  
13 using std::string;
```

```

14
15  /* ===== */
16  /* Names of malloc and free */
17  /* ===== */
18  #if defined(TARGET_MAC)
19  #define MALLOC "_malloc"
20  #define FREE "_free"
21  #else
22  #define MALLOC "malloc"
23  #define FREE "free"
24  #endif
25
26  /* ===== */
27  /* Global Variables */
28  /* ===== */
29
30  std::ofstream TraceFile;
31
32  /* ===== */
33  /* Commandline Switches */
34  /* ===== */
35
36  KNOB< string > KnobOutputFile(KNOB_MODE_WRITEONCE, "pintool", "o",
    "malloctrace.out", "specify trace file name");
37
38  /* ===== */
39
40  /* ===== */
41  /* Analysis routines */
42  /* ===== */
43
44  VOID Arg1Before(CHAR* name, ADDRINT size) { TraceFile << name << "(" << size
    << ")" << endl; }
45
46  VOID MallocAfter(ADDRINT ret) { TraceFile << " returns " << ret << endl; }
47
48  /* ===== */
49  /* Instrumentation routines */
50  /* ===== */
51
52  VOID Image(IMG img, VOID* v)
53  {
54      // Instrument the malloc() and free() functions. Print the input argument
55      // of each malloc() or free(), and the return value of malloc().
56      //
57      // Find the malloc() function.
58      RTN mallocRtn = RTN_FindByName(img, MALLOC);

```

```

59     if (RTN_Valid(mallocRtn))
60     {
61         RTN_Open(mallocRtn);
62
63         // Instrument malloc() to print the input argument value and the
return value.
64         RTN_InsertCall(mallocRtn, IPOINT_BEFORE, (AFUNPTR)Arg1Before,
IARG_ADDDRINT, MALLOC, IARG_FUNCARG_ENTRYPOINT_VALUE, 0,
65                     IARG_END);
66         RTN_InsertCall(mallocRtn, IPOINT_AFTER, (AFUNPTR)MallocAfter,
IARG_FUNCRET_EXITPOINT_VALUE, IARG_END);
67
68         RTN_Close(mallocRtn);
69     }
70
71     // Find the free() function.
72     RTN freeRtn = RTN_FindByName(img, FREE);
73     if (RTN_Valid(freeRtn))
74     {
75         RTN_Open(freeRtn);
76         // Instrument free() to print the input argument value.
77         RTN_InsertCall(freeRtn, IPOINT_BEFORE, (AFUNPTR)Arg1Before,
IARG_ADDDRINT, FREE, IARG_FUNCARG_ENTRYPOINT_VALUE, 0,
78                     IARG_END);
79         RTN_Close(freeRtn);
80     }
81 }
82

```

上面是这个工具的框架描述，下面我们来总结一般的Pintool基本框架：

1. 头文件引入与初始化

Pintool 程序以包含 `pin.H` 头文件开头，这是使用 Pin 插桩功能所必需的头文件。主函数 `main` 中首先调用 `PIN_Init()` 来初始化 Pin 工具，并解析命令行参数。

2. 插桩函数定义 (Instrumentation Functions)

Pintool 的核心在于“插桩”目标程序。开发者需要定义一个或多个插桩函数，这些函数会在目标程序运行期间，根据指定规则被 Pin 框架插入到程序的不同位置。

3. 分析函数定义 (Analysis Functions)

插桩本身并不执行实际的分析逻辑，而是通过 `INS_InsertCall` 或 `RTN_InsertCall` 将自定义的分析函数插入到目标程序执行流中。

4. 程序退出处理 (Fini 函数)

Pintool 还支持定义一个退出回调函数 `PIN_AddFiniFunction`，即目标程序结束运行前自动执行的清理逻辑，通常用来：

- 输出最终统计结果；
- 关闭打开的日志文件；
- 释放内存资源等。

5. 启动目标程序

在完成插桩规则定义后，调用 `PIN_StartProgram()` 启动目标程序的实际执行。

四、实验总结

- 在本次实验中，我们基于 Intel 的 Pin 工具在 Kali 虚拟机中成功复现了 `malloctrace` 插桩工具，深入理解了 Pintool 插桩的基本流程和使用方法。
 - 实验过程中，完成了对 Pintool 的编译、插桩、运行与结果分析，掌握了 `malloc` 和 `free` 函数调用的拦截与跟踪方式，并能通过日志输出文件查看内存分配与释放的细节。
 - 借助 `malloctrace.cpp` 的源码，我们分析并总结了 Pintool 插桩工具的基本框架，包括分析函数、插桩函数、程序映像处理、程序退出清理等核心组成部分。
 - 此外，通过实际在 Kali 系统下操作，更加熟悉了 Linux 环境下的命令行使用及工具配置流程，对动态插桩和二进制分析的原理有了更清晰的认识，也为后续的二进制安全分析和渗透测试打下了基础。
-